

Question 1. Answer the following questions regarding the fundamental concepts of object-oriented approaches.

1. In Object-Oriented Programming, what does encapsulation mainly achieve in a class like BankAccount where balance is private and accessed only through methods like deposit() and getBalance()?

- a) It allows creating and using an object in a single line of code.
- b) It restricts direct access to internal data and provides controlled access through methods.
- c) It forces the class to be declared as final and non-extendable.
- d) It removes the need to define class attributes separately from methods.

2. Consider a class named Student in Java that does NOT explicitly declare any constructors. Which of the following statements correctly shows how to create an object of this class?

- a) `new Student();`
- b) You must define a constructor before creating an object.
- c) `new Object("Student");`
- d) `new Student(1);`

3. Given the following code:

```
Student s1 = new Student("Ali");  
Student s2 = new Student("Ali");  
Student s3 = s1;
```

- a) `s1 == s2` and `s1 == s3` are both true.
- b) `s1 == s2` is true, but `s1 == s3` is false.
- c) `s1 == s3` is true, but `s1 == s2` is false.
- d) All three references point to the same object.

4. Which of the following correctly describes the relationship between abstract classes and interfaces in Java?

- a) A class can extend multiple abstract classes but implement only one interface.
- b) An abstract class must implement all methods from an interface.
- c) An interface can contain constructors if declared abstract.
- d) A class can implement multiple interfaces but extend only one abstract class.

5. The following Java program models course registrations in a university system. After executing the code, draw the final memory state, showing how each reference (c1, c2, c3, and c4) is related to the created objects.

```
class Course {
    String name;
    public Course(String name) {
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
        Course c1 = new Course("Math");
        Course c2 = new Course("Physics");
        Course c3 = c1;
        Course c4 = null;
    }
}
```

6. A mobile app assigns a unique ID to each new user when they register. What is the output of the following code that simulates this behavior?

```
public class User {
    static int totalUsers = 0;
    int userId;
    public User() {
        totalUsers++;
        userId = totalUsers;
    }
    public static void main(String[] args) {
        User u1 = new User();
        User u2 = new User();
        User u3 = new User();
        System.out.println("Total Registered Users: " + User.totalUsers);
        System.out.println("User ID of u2: " + u2.userId);
    }
}
```

Question 2. Answer the following questions regarding the relationship between object-oriented inheritance and subclasses.

1. Assume both classes Vehicle and Car are in the same package. What is the output of the following code? [1Mark]

```
public class Vehicle {
    protected String log = "";
    public Vehicle(String type) {
        log += type + "-";
    }
    public String getLog() {
        return log;
    }

    public static void main(String[] args) {
        Car myCar = new Car();
        System.out.print(myCar.getLog());
        System.out.println(myCar.getType());
    }

    public class Car extends Vehicle {
        String type = "Car";
        public Car() {
            super("Vehicle");
            log += "Drive-";
        }
        public String getType() {
            return type;
        }
    }
}
```

2. What is the output of the following code?

```
public class Animal {
    String name = "Unknown";
    public String getDetails() {
        return "Animal: " + name;
    }
}

public class Dog extends Animal {
    String name = "Dog";
    public String getDetails() {
        return "Dog: " + name + ", " + super.getDetails();
    }
}

public class Puppy extends Dog {
    String name = "Puppy";
    public String getDetails() {
        return "Puppy: " + name + ", " + super.getDetails();
    }
}

public class Main {
    public static void main(String[] args) {
        Puppy p = new Puppy();
        System.out.println(p.getDetails());  }}
```

3. Assume you have two classes: Device and Mobile. Device is defined in package A and contains two data fields: protected int x; and int y; (y is default). Mobile is defined in package B and extends Device.

What does this inheritance relationship allow the Mobile class to do?

- a) Mobile can directly access both x and y.
- b) Mobile can directly access only x.
- c) Mobile can directly access only y.
- d) Mobile cannot directly access either x or y.

Consider the following Java class:

```
public final class Employee {  
    private String name;  
    private ArrayList<String> skills;  
  
    public Employee(String name, ArrayList<String> skills) {  
        this.name = name;  
        this.skills = skills;  
    }  
  
    public String getName() {  
        return name;  
    }  
  
    public ArrayList<String> getSkills() {  
        return skills;  
    }  
}
```

4. Which of the following statements is true about the Employee class?

- a) The class is immutable because all fields are private and final, and no setters are defined.
- b) The class is immutable because it cannot be subclassed, and fields cannot be reassigned.
- c) The class is mutable because it returns a direct reference to a mutable object.
- d) None of the above

5. What is the output of the following Java program?

```
import java.util.Random;  
public class ManualShuffle {  
    public static void main(String[] args) {  
        int[] arr = {1, 2, 3};  
        Random r1 = new Random(123);  
        Random r2 = new Random(123);  
        for (int i = 1000; i > 0; i--) {  
            int j = r2.nextInt(arr.length);  
            int k = r2.nextInt(arr.length);  
            int temp = arr[j];  
            arr[j] = arr[k];  
            arr[k] = temp;  
        }  
        for (int i = 0; i < arr.length; i++) {  
            System.out.print(arr[i] + " ");  
        }  
    }  
}
```

Question 3. Answer the following question regarding interfaces and abstract classes.

1. Consider the following code involving interfaces and abstract classes from a smart home device system:

```
interface Controllable {
    void turnOn();
}

abstract class SmartDevice implements Controllable {
    abstract void configure();
}

public class SmartLamp extends SmartDevice {
    void configure() {
        System.out.println("Configuring Smart Lamp");
    }

    public void turnOn() {
        System.out.println("Turning on Smart Lamp");
    }

    public static void main(String[] args) {
        // Evaluate the following code snippets...
    }
}
```

Based on the above code, Indicate whether the following code snippets are correct or not. If correct, show the output. [0.5Markforeachanswer]

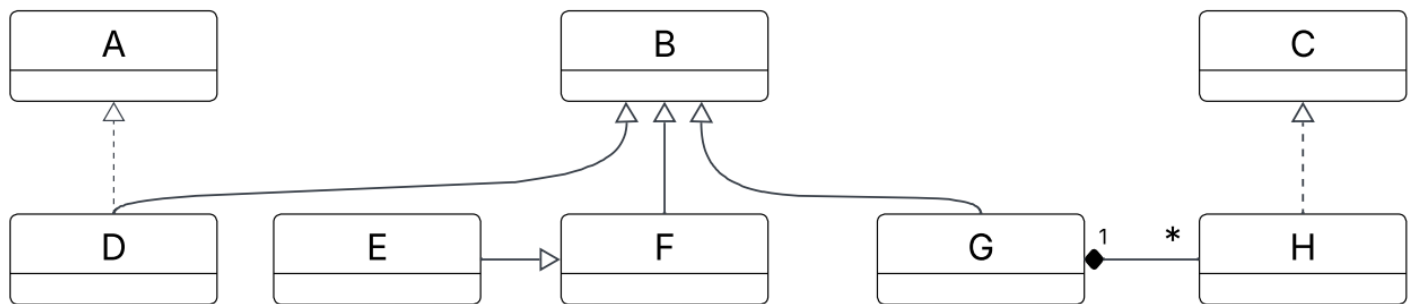
Code Snippet		Correct / Not Correct	Output if Correct
1	SmartLamp lamp = new SmartLamp(); lamp.configure();		
2	SmartDevice sd = new SmartLamp(); sd.turnOn();		
3	Controllable c = new SmartLamp(); c.configure();		
4	Controllable c = new SmartLamp(); if (c instanceof SmartLamp) ((SmartLamp)c).configure();		
5	Controllable ctrl = new SmartLamp(); ctrl.turnOn();		

Question 4. Answer the following questions based on your understanding of the given UML diagram.

1. In the given table in the answer sheet, identify the matching block (from A to H) for each of the following classes, abstract classes, and interfaces based on the UML diagram and the descriptions of the relationships between them as follows:

#	Name	Type	UML block ID (from A to H)
1)	Row	Class	
2)	Text	Class	
3)	Image	Class	
4)	Table	Class	
5)	Graphic	Abstract Class	
6)	WebContent	Abstract Class	
7)	Translatable	interface	
8)	Sortable	interface	

UML diagram:

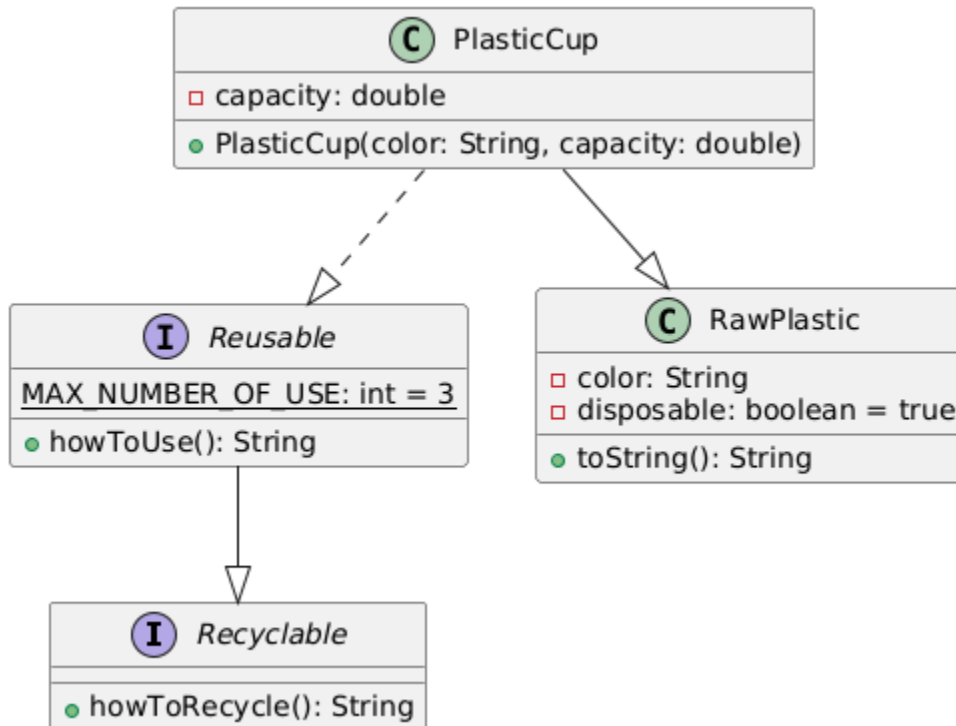


UML Descriptions:

- An **Image** is a **Graphic**.
- **Graphic** is a **WebContent**.
- A **Text** is a **WebContent**.
- A **Text** is an instance of **Translatable**.
- A **Table** is a **WebContent**.
- A **Table** has one or more **Rows**.
- A **Row** is an instance of **Sortable**.

Question 5. Answer the following question regarding the analysis of a real-world situation in an object-oriented way.

1. Write the code segments for a manufacturer of plastic products by implementing only the required class and interface below, based on the following UML diagram.



Note:

- indicates public access.
- indicates private access.

A. Interface: Reusable

- Declare the **Reusable** interface with its constant and method, considering its relationship to other components as shown in the UML diagram.

B. Class: PlasticCup

- Create the **PlasticCup** class based on its relationships to other classes or interfaces and declare its variable and constructor, as shown in the UML diagram above.
- Implement all methods that **must be implemented** in the **PlasticCup** class based on its inherited class and implemented interface, as in the UML diagram above.

(**Note:** use the string "**Melting**" to describe how a plastic cup is recycled, and use the string "**Serving drinks**" to describe how a plastic cup is used)

Question 6. Answer the following questions regarding your understanding of ArrayList, Exception Handling, Generics and Polymorphism.

1. You are given a class called Container that is designed to store a single object. Currently, this container is designed to only store a Car object, like this:

```
class Car {}
class Container {
    private Car item;
    public void set(Car item) {
        this.item = item;
    }
    public Car get() {
        return item;
    }
}
```

A. Explain the limitation of the current Container class design. Why can it not store objects of other types like Truck or Computer? [1 mark]

B. We want to redesign the Container class so that it can be used to store any object type, not Just Car. The goal is to: [2 marks]

- Make the container reusable for different types (Car, Truck, Computer, etc.)
- Avoid the need for **casting** when retrieving the object
- Enforce **compile-time type safety**

2. What is the output when the following code is executed? [2 marks]

```
public class FinalExam_2025_2 {
    public static void main(String[] args) {
        try {
            m1();
        } catch (ArithmeticException e) {
            System.out.println(e.getMessage());
            System.out.println("Main");
        }
    }
    public static void m1() {
        System.out.println("In m1");
        m2();
    }
    public static void m2() {
        try {
            m3();
        } catch (IllegalArgumentException e) {
```



```

        System.out.println(e.getMessage());
        System.out.println("m2");
    } finally {
        System.out.println("Finally");
    }
}
public static void m3() {
    throw new ArithmeticException("Error in m3");
}}

```

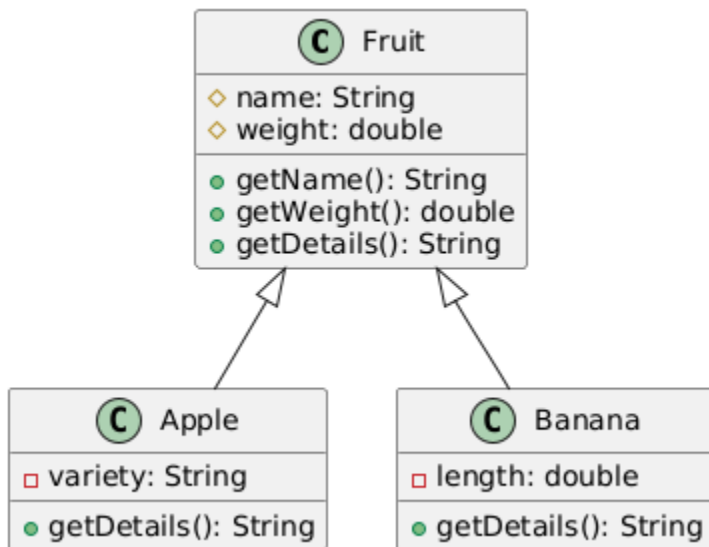
3. As shown in the diagram, the class **Fruit** includes basic information and a method called **getDetails()**, which is overridden in each subclass.

You are given the following code:

```

Apple v1 = new Apple("Red Delicious", 150, "Sweet");
Apple v2 = new Apple("Granny Smith", 160, "Tart");
Banana v3 = new Banana("Cavendish", 120, 18.5);
Banana v4 = new Banana("Plantain", 200, 22.0);

```



Note:

◇ indicates protected access.

● indicates public access.

◻ indicates private access.

A. Declare an ArrayList that holds objects of type Fruit. [1 mark]

B. Add the four given objects (v1, v2, v3, v4) to the ArrayList you created. [2 marks]

C. Write a for loop that iterates over the ArrayList and prints the details of only the Apple objects using the getDetails() method. [2 marks]

